

# 8

## Реализация основных моделей в дискретно-событийном подходе

### 8.1 Постановка задачи

#### 8.1.1 Цель

Изучить дискретно-событийный подход к имитационному моделированию на примере классической модели распространения инфекции SIR. Реализовать стохастическую дискретно-событийную модель в виде программного комплекса на языке Julia. Провести анализ влияния параметров, сравнить со стохастической и детерминированной версиями, оценить производительность и модифицировать модель.

### 8.2 Программный комплекс

Программный комплекс состоит из двух основных файлов:

- `src/sir_model.jl` (ядро модели): определение структур данных, функций событий, логики агентов и управления симуляцией.
- `scripts/sir_des.jl` (скрипт запуска): задание параметров, инициализация модели, выполнение прогона и визуализация.

### 8.2.1 Используемые внешние библиотеки

- `ResumableFunctions`: создание возобновляемых функций-генераторов с помощью макроса `@resumable` и оператора `@yield`.
- `ConcurrentSim`: ядро дискретно-событийного моделирования: управление виртуальным временем, планирование событий через `timeout` и запуск процессов через `@process`.
- `Distributions`: генерация случайных величин (экспоненциальное, дискретное равномерное, равномерное распределения).
- `DataFrames`: формирование таблицы результатов.
- `StatsPlots`: построение графиков.

### 8.2.2 Структура программы (`sir_model.jl`)

#### 8.2.2.1 Вспомогательные функции мутации массивов статистики

- `increment!(a::Array{Int64})` — добавляет в конец массива значение, увеличенное на 1 относительно последнего элемента.
- `decrement!(a::Array{Int64})` — добавляет значение, уменьшенное на 1.
- `carryover!(a::Array{Int64})` — дублирует последнее значение.

Эти функции используются для фиксации изменений численности  $S$ ,  $I$ ,  $R$  в момент наступления события. Массивы  $S_a$ ,  $I_a$ ,  $R_a$  хранят временные ряды численности: каждая операция добавляет новую точку, соответствующую текущему модельно времени.

#### 8.2.2.2 Определение структур данных

- `SIRPerson` Агент-индивид:
  - `id::Int64` — уникальный идентификатор.
  - `status::Symbol` — текущее состояние: `:S`, `:I` или `:R`.
- `SIRModel` Хранит всё состояние модели:
  - `sim::ConcurrentSim.Simulation` — объект симуляции (виртуальное время и очередь событий).
  - `β::Float64` — вероятность заражения при контакте с инфицированным.
  - `c::Float64` — частота контактов (среднее число контактов в единицу времени для одного индивида).

- `γ::Float64` – интенсивность выздоровления ( $1 /$  средняя длительность болезни).
- `ta::Array{Float64}` – временной ряд: моменты событий.
- `Sa::Array{Int64}` – временной ряд численности восприимчивых.
- `Ia::Array{Int64}` – временной ряд численности инфицированных.
- `Ra::Array{Int64}` – временной ряд численности выздоровевших.
- `allIndividuals::Array{SIRPerson}` – список всех индивидов.

### 8.2.2.3 Функции обновления статистики при событиях

- `infection_update!(sim::ConcurrentSim.Simulation, m::SIRModel)` – вызывается в момент заражения. Добавляет текущее время `now(sim)` в `ta`, уменьшает число `S` (`decrement!(m.Sa)`), увеличивает `I` (`increment!(m.Ia)`), дублирует `R` (`carryover!(m.Ra)`).
- `recovery_update!(sim::ConcurrentSim.Simulation, m::SIRModel)` – вызывается при выздоровлении. Добавляет время, оставляет `S` без изменений (`carryover!`), уменьшает `I`, увеличивает `R`.

### 8.2.2.4 Основной процесс агента: функция `live`

Функция `live` описывает жизненный цикл индивида. Она размечена макросом `@resumable`, что позволяет приостанавливать её выполнение с помощью `@yield timeout(...)`, не блокируя поток.

Логика работы:

- Пока индивид находится в состоянии `:S`:
  - Ожидает случайное время, распределённое экспоненциально с параметром  $1/c$ . Это моделирует промежуток до следующего социального контакта.
  - После ожидания выбирает случайного собеседника (`alter`) из всей популяции (исключая себя).
  - Если собеседник в состоянии `:I`, то с вероятностью  $\beta$  происходит заражение: статус меняется на `:I` и вызывается `infection_update!`.
- Если индивид стал инфицированным (`:I`):
  - Ожидает время до выздоровления, распределённое экспоненциально с параметром  $1/\gamma$ .
  - После этого статус меняется на `:R`, и вызывается `recovery_update!`.

Благодаря конкурентному выполнению всех агентов (каждый запущен как отдель-

ный процесс), симуляция естественно воспроизводит стохастическую динамику эпидемии.

### 8.2.2.5 Функции управления моделью

- `MakeSIRModel(u0, p)` Создает экземпляр `SIRModel`. Вход: начальные условия  $u0 = [S0, I0, R0]$  и параметры  $p = [\beta, c, \gamma]$ . Процесс:
  - Вычисляет общую популяцию  $N = S0 + I0 + R0$ .
  - Создает объект `ConcurrentSim.Simulation()`.
  - Генерирует массив индивидов `allIndividuals` с соответствующими статусами.
  - Инициализирует временные ряды  $ta = [0.0]$ ,  $Sa = [S0]$ ,  $Ia = [I0]$ ,  $Ra = [R0]$ .
  - Возвращает готовую модель.
- `activate(m::SIRModel)` Запускает все процессы агентов: для каждого индивида создается `@process live(m.sim, individual, m)`. На этом этапе процессы ещё не выполняются – они только регистрируются в симуляции.
- `sir_run(m::SIRModel, tf::Float64)` Запускает симуляцию: `ConcurrentSim.run(m.sim, tf)`, где  $tf$  – конечное модельное время. Виртуальное время продвигается от события к событию, выполняются все запланированные `timeout`.
- `out(m::SIRModel)` Собирает результаты в `DataFrame` с колонками `:t`, `:S`, `:I`, `:R`.

---

```
using ResumableFunctions, ConcurrentSim, Distributions, DataFrames,
    Random
```

```
# Вспомогательные функции для обновления массивов состояния
```

```
function increment!(a::Array{Int64})
    push!(a, a[length(a)] + 1)
```

```
end
```

```
function decrement!(a::Array{Int64})
    push!(a, a[length(a)] - 1)
```

```
end
```

```
function carryover!(a::Array{Int64})
    push!(a, a[length(a)])
```

```
end
```

```
# Структуры данных
```

```
mutable struct SIRPerson
    id::Int64
```

```

    status::Symbol # :S, :I, :R
end

mutable struct SIRModel
    sim::ConcurrentSim.Simulation # Тип Simulation, не Environment
    β::Float64
    c::Float64
    γ::Float64
    ta::Array{Float64}
    Sa::Array{Int64}
    Ia::Array{Int64}
    Ra::Array{Int64}
    allIndividuals::Array{SIRPerson}
end

# Функции обновления статистики при событиях
function infection_update!(sim::ConcurrentSim.Simulation,
    ↪ m::SIRModel)
    push!(m.ta, ConcurrentSim.now(sim))
    decrement!(m.Sa)
    increment!(m.Ia)
    carryover!(m.Ra)
end

function recovery_update!(sim::ConcurrentSim.Simulation, m::SIRModel)
    push!(m.ta, ConcurrentSim.now(sim))
    carryover!(m.Sa)
    decrement!(m.Ia)
    increment!(m.Ra)
end

# Основная логика жизни индивида
@resumable function live(env::ConcurrentSim.Simulation,
    ↪ individual::SIRPerson, m::SIRModel)
    while individual.status == :S
        @yield timeout(env, rand(Exponential(1/m.c)))
        alter = individual
        while alter == individual
            N = length(m.allIndividuals)
            index = rand(DiscreteUniform(1, N))
            alter = m.allIndividuals[index]
        end
        if alter.status == :I
            if rand(Uniform(0, 1)) < m.β

```

```

        individual.status = :I
        infection_update!(env, m)
    end
end
end
if individual.status == :I
    @yield timeout(env, rand(Exponential(1/m.γ)))
    individual.status = :R
    recovery_update!(env, m)
end
end

# Функции создания и запуска модели
function MakeSIRModel(u0, p)
    (S, I, R) = u0
    N = S + I + R
    (β, c, γ) = p
    sim = ConcurrentSim.Simulation() # Создаём именно Simulation
    allIndividuals = SIRPerson[]
    for i = 1:S
        push!(allIndividuals, SIRPerson(i, :S))
    end
    for i = (S+1):(S+I)
        push!(allIndividuals, SIRPerson(i, :I))
    end
    for i = (S+I+1):N
        push!(allIndividuals, SIRPerson(i, :R))
    end
    ta = Float64[0.0]
    Sa = Int64[S]
    Ia = Int64[I]
    Ra = Int64[R]
    SIRModel(sim, β, c, γ, ta, Sa, Ia, Ra, allIndividuals)
end

function activate(m::SIRModel)
    [@process live(m.sim, individual, m) for individual in
     ↪ m.allIndividuals]
end

function sir_run(m::SIRModel, tf::Float64)
    ConcurrentSim.run(m.sim, tf)
end

```

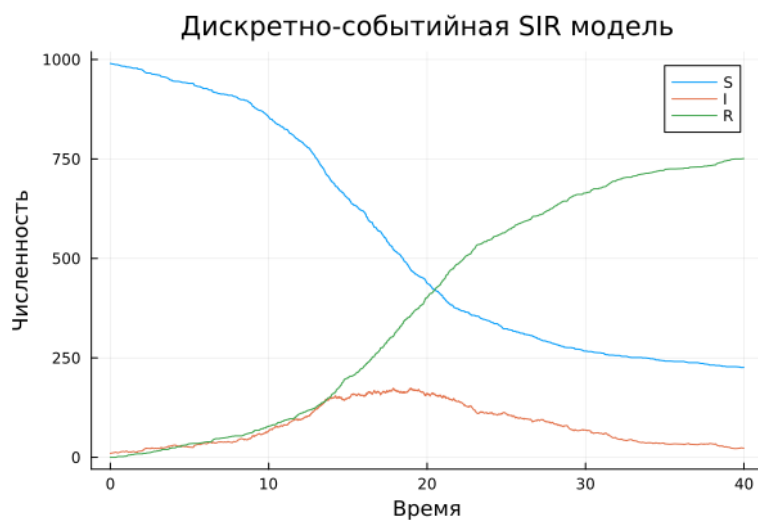
```

function out(m::SIRModel)
    result = DataFrame()
    result[:, :t] = m.ta
    result[:, :S] = m.Sa
    result[:, :I] = m.Ia
    result[:, :R] = m.Ra
    return result
end

```

### 8.2.3 Скрипт запуска (sir\_des.jl)

- С помощью `@quickactivate "project"` активируется окружение проекта.
- Подключается `sir_model.jl` и необходимые пакеты.
- Задаются входные параметры:
  - `tmax = 40.0` — длительность симуляции.
  - `u0 = [990, 10, 0]` — начальные количества S, I, R.
  - `p = [0.05, 10.0, 0.25]` —  $\beta$ ,  $c$ ,  $\gamma$ .
- Фиксируется зерно генератора случайных чисел для воспроизводимости.
- Создаётся модель, активируются процессы, выполняется прогон.
- Результат преобразуется в `DataFrame`.
- Строится график с кривыми S, I, R и сохраняется в `plots/sir_des.png` (рис. 8.1).



**Рисунок 8.1.** График SIR

```
using DrWatson
```

```
@quickactivate "project"
include(sourcedir("sir_model.jl"))
using Random, StatsPlots, BenchmarkTools

# Параметры модели
tmax = 40.0
u0 = [990, 10, 0]      # S, I, R
p = [0.05, 10.0, 0.25] #  $\beta$ , c,  $\gamma$ 

Random.seed!(1234)

# Запуск модели
des_model = MakeSIRModel(u0, p)
activate(des_model)
sir_run(des_model, tmax)
data_des = out(des_model)

# Визуализация
@df data_des plot(
    :t,
    [:S :I :R],
    labels = ["S" "I" "R"],
    xlab = "Время",
    ylab = "Численность",
    title = "Дискретно-событийная SIR модель",
)
savefig(plotsdir("sir_des.png"))
```

---

## 8.3 Входные и выходные данные

### 8.3.1 Входные данные

- Начальное состояние популяции  $u_0$  (вектор из трёх целых чисел): число восприимчивых, инфицированных и переболевших на момент времени 0.
- Параметры модели  $p$  (вектор из трёх вещественных чисел):
  - $\beta$  (вероятность передачи при контакте) – число от 0 до 1.
  - $c$  (частота контактов) – среднее число контактов индивида в единицу модельного времени.
  - $\gamma$  (интенсивность выздоровления) – величина  $> 0$ ; обратная к среднему



времени болезни.

- Длительность симуляции `tmax` (вещественное число) – момент остановки модельного времени.
- Зерно генератора случайных чисел – опционально, для воспроизводимости.

### 8.3.2 Выходные данные

- Таблица `DataFrame` с колонками:
  - `t` – моменты времени наступления событий.
  - `S` – число восприимчивых.
  - `I` – число инфицированных.
  - `R` – число переболевших.
- График временных рядов `S`, `I`, `R`, сохранённый в `plots/sir_des.png`.

## 8.4 Общий алгоритм

Программа моделирует стохастическое распространение инфекции в полностью смешивающейся популяции конечного размера. В отличие от детерминированной системы ОДУ, здесь наблюдаются случайные флуктуации, а также возможны ранние вымирания от инфекции при малых начальных `I`.

- Инициализация: создание объектов индивидов с начальными статусами, инициализация статистических массивов.
- Планирование процессов: для каждого индивида создаётся параллельный процесс (`@process`), реализующий его жизненный цикл.
- Запуск симуляции: `ConcurrentSim.run` продвигает виртуальное время, обрабатывая события в хронологическом порядке. Каждый `timeout` добавляет событие в календарь; когда время события наступает, выполняется соответствующий код внутри `live` (возможно, порождая новые события).
- Сбор статистики: в момент изменения состояния индивида обновляются временные ряды `Sa`, `Ia`, `Ra`, а также запоминается текущее время.
- Завершение: по достижении `tf` симуляция останавливается. Все накопленные ряды экспортируются в таблицу.
- Постобработка: построение графиков, сохранение результатов.

## 8.5 Дополнительные задания

При выполнении лабораторной работы предлагается модифицировать программу для углублённого изучения.

### 8.5.1 Анализ чувствительности к параметрам

#### 8.5.1.1 Описание

Провести несколько прогонов с разными значениями  $\beta$ ,  $c$ ,  $\gamma$ . Для каждого набора параметров построить график или записать ключевые метрики (пик  $I$ , время пика, итоговая доля переболевших). Объяснить, как изменение каждого параметра влияет на динамику.

#### 8.5.1.2 Что нужно сделать в коде

Написать цикл, варьирующий параметры и сохраняющий результаты, например:

---

```
betas = [0.03, 0.05, 0.07]
for  $\beta$  in betas
    p = [ $\beta$ , 10.0, 0.25]
    m = MakeSIRModel(u0, p)
    activate(m)
    sir_run(m, tmax)
    data = out(m)
    # построить график или сохранить метрики
end
```

---

### 8.5.2 Детерминированная длительность болезни

#### 8.5.2.1 Описание

Заменить экспоненциальное время выздоровления на фиксированную величину  $1/\gamma$ . Сравнить со стохастической версией. Обратить внимание на отсутствие

«хвостов» распределения и более синхронное выздоровление.

### 8.5.2.2 Модификация кода

В `src/sir_model.jl` в функции `live` заменить строку

---

```
@yield timeout(env, rand(Exponential(1/m.γ)))
```

---

на

---

```
@yield timeout(env, 1/m.γ)
```

---

Запустить оба варианта с одинаковым зерном. Наложить графики.

## 8.5.3 Оценка производительности

### 8.5.3.1 Описание

С помощью макроса `@benchmark` измерить время выполнения `sir_run` для популяции 10000 индивидов. Предложить способы оптимизации (например, использование векторизованных операций или предварительное генерирование случайных чисел).

### 8.5.3.2 Модификация кода

Добавить в скрипт:

---

```
using BenchmarkTools
@benchmark sir_run(des_model, tmax)
```

---

Проанализировать отчёт.

## 8.5.4 Сохранение результатов в CSV

### 8.5.4.1 Описание

Добавить автоматическое сохранение итоговой таблицы в каталог `data/sims/` с уникальным именем, содержащим параметры запуска.

### 8.5.4.2 Модификация кода

Добавить в `sir_des.jl`:

---

```
using CSV, Dates
filename = "sir_$(u0[1])_$(u0[2])_$(p[1])_$(p[2])_$(p[3]).csv"
CSV.write(datadir("sims", filename), data_des)
```

---

## 8.5.5 Добавление демографических событий

### 8.5.5.1 Описание

Расширить модель, добавив в жизненный цикл индивида возможность смерти (с постоянной интенсивностью  $\mu$ ) и рождение новых восприимчивых. Смерть удаляет индивида из популяции, рождение добавляет нового в состояние :S. Это позволяет исследовать эндемическое равновесие.

### 8.5.5.2 Модификация кода

В функцию `live` добавить блок ожидания смерти:

---

```
@yield timeout(env, rand(Exponential(1/μ)))
```

---

- Смерть наступает: статус меняется на :R (или новое состояние :D).
- Нужно также обновить статистику: уменьшить S, I или R в зависимости от статуса.

- Для рождения можно создать отдельный процесс, порождающий новых индивидов с постоянной интенсивностью.

### 8.5.6 Вакцинация

#### 8.5.6.1 Описание

Реализовать стратегию вакцинации: в определённый момент времени (или при превышении порога инфицированных) часть восприимчивых мгновенно переводится в :R. Исследовать, какая доля вакцинированных необходима для предотвращения эпидемии.

#### 8.5.6.2 Модификация кода

Добавить в модель срабатывание события вакцинации по таймеру:

```
@resumable function vaccinate(env, m, time, fraction)
  @yield timeout(env, time)
  # переводим fraction*S индивидов в :R, обновляем статистику
end
```

### 8.5.7 Модель SEIR

#### 8.5.7.1 Описание

Ввести латентный период (статус :E). После заражения индивид переходит в :E и через время, распределённое экспоненциально с параметром  $\sigma$ , становится инфекционным (:I). Оценить, как меняется динамика по сравнению с SIR.

#### 8.5.7.2 Модификация кода

Добавить состояние :E. В live после заражения переводить в :E и планировать переход в :I через `@yield timeout(env, rand(Exponential(1/σ)))`.

## 8.6 Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
  - чистый код;
  - jupyter notebook;
  - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:
  - чистый код;
  - jupyter notebook;
  - документацию в формате Quarto.
- Выполнить код из jupyter notebook с параметрами.
- Интегрировать документацию с параметрами в формате Quarto в отчёт.